

Wazuh SIEM Home Lab

Attack Simulation, Detection, and Automated Response: Project Walkthrough

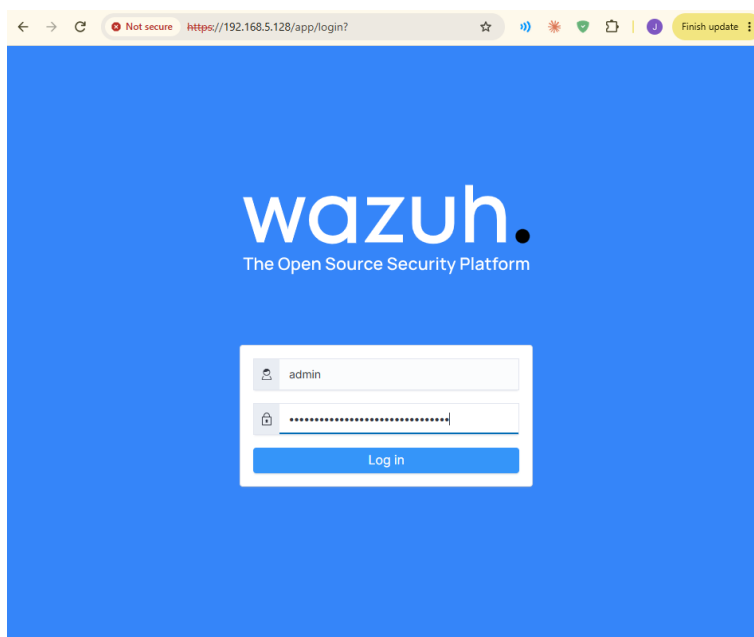
This project involved building a Wazuh SIEM (Security Information and Event Management) lab on an Ubuntu server, then testing it by simulating real attacker behavior with Atomic Red Team, a library of tests mapped to the MITRE ATT&CK framework. The goal was to confirm Wazuh could actually detect that activity, correctly tag it against known attack techniques, and, as a final step, automatically respond to a real threat by detecting and removing a malicious file using a VirusTotal integration.

1. Installing Wazuh

Wazuh was installed on an Ubuntu server, which bundles three components: the manager (processes and analyzes logs), the indexer (stores alert data), and the dashboard (the web interface used to view everything). Once the install finished, it printed the dashboard's URL along with a generated admin username and password.

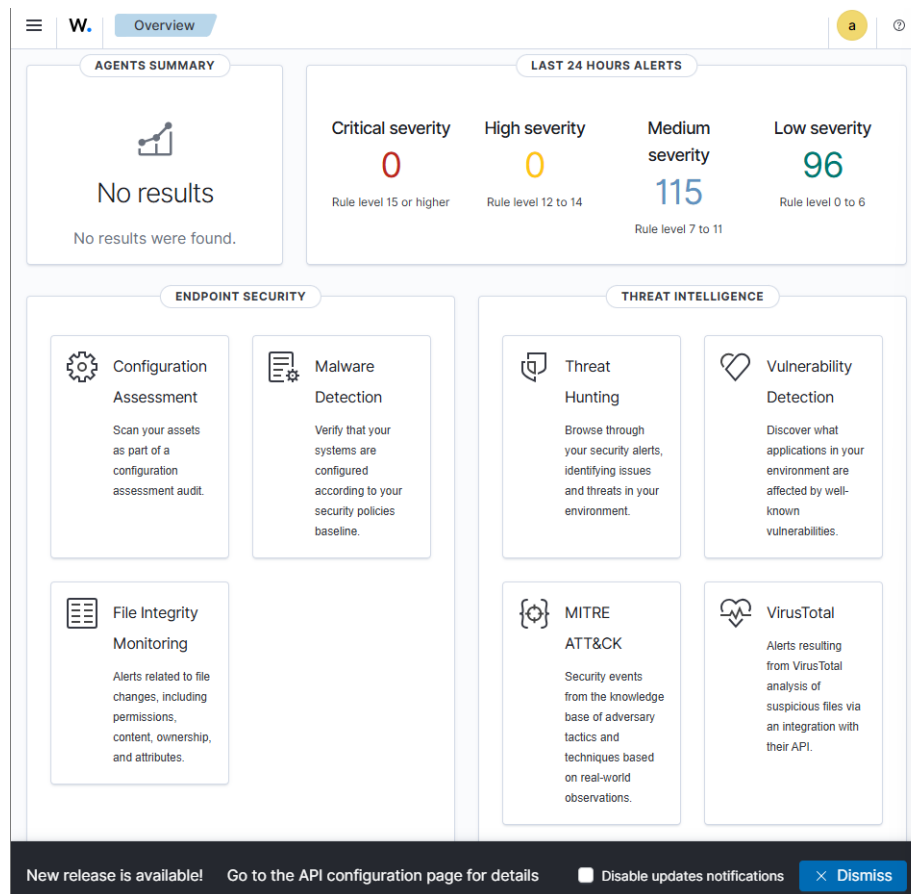
```
15/08/2026 15:53:29 INFO: wazuh-dashboard web application initialized.
15/08/2026 15:53:29 INFO: --- Summary ---
15/08/2026 15:53:29 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
User: admin
Password: UCfnayyQK?cN5BfE3o9eP9cIdYRyz4uJ
15/08/2026 15:53:30 INFO: Installation finished.
josh@wazuh-server:~$
```

Using those credentials, I logged into the Wazuh dashboard from a browser.



2. The Dashboard

The Overview page is the main landing screen. It shows a summary of connected agents and a breakdown of alerts from the last 24 hours by severity, along with shortcuts to Wazuh's individual security modules, including File Integrity Monitoring, MITRE ATT&CK, and VirusTotal, all used later in this project.



3. Setting Up Atomic Red Team

To actually test whether Wazuh detects attacks, I used Atomic Red Team, an open-source library of small, safe scripts that each simulate one specific technique from the MITRE ATT&CK framework (a standard catalog of real-world attacker behavior).

```
josh@wazuh-server:~$ sudo apt install git -y
git clone https://github.com/redcanaryco/atomic-red-team.git
cd atomic-red-team/atomics
bash T1046/T1046.sh
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.34.1-1ubuntu1.17).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 63 not upgraded.
Cloning into 'atomic-red-team'...
remote: Enumerating objects: 63674, done.
remote: Counting objects: 100% (108/108), done.
remote: Compressing objects: 100% (50/50), done.
Receiving objects: 14% (8915/63674), 49.75 MiB | 10.59 MiB/s
```

Rather than manually running each test, I set up PowerShell and installed the Invoke-AtomicRedTeam module, which runs a given technique end-to-end: it installs anything it needs first, then executes the actual simulated attack.

```

2026-08-15 16:57:03 (532 MB/s) - 'packages-microsoft-prod.deb' saved [4070/4070]

josh@wazuh-server:~/atomic-red-team/atomics$ sudo dpkg -i packages-microsoft-prod.deb
Selecting previously unselected package packages-microsoft-prod.
(Reading database ... 203516 files and directories currently installed.)
Preparing to unpack packages-microsoft-prod.deb ...
Unpacking packages-microsoft-prod (1.0-ubuntu22.04.1) ...
Setting up packages-microsoft-prod (1.0-ubuntu22.04.1) ...
josh@wazuh-server:~/atomic-red-team/atomics$ sudo apt update
sudo apt install -y powershell
Hit:1 http://us.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 https://packages.wazuh.com/4.x/apt stable InRelease
Get:4 https://packages.microsoft.com/ubuntu/22.04/prod jammy InRelease [3,632 B]
Hit:5 http://us.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:6 http://security.ubuntu.com/ubuntu jammy-security InRelease
Get:7 https://packages.microsoft.com/ubuntu/22.04/prod jammy/main all Packages [1,333 B]
Get:8 https://packages.microsoft.com/ubuntu/22.04/prod jammy/main arm64 Packages [373 kB]
Get:9 https://packages.microsoft.com/ubuntu/22.04/prod jammy/main amd64 Packages [598 kB]

```

4. Simulating an Attack: Network Service Discovery (T1046)

The first test simulated MITRE technique T1046, Network Service Discovery, where an attacker scans a network to find open ports and running services, typically to map out what else is reachable after gaining initial access.

```

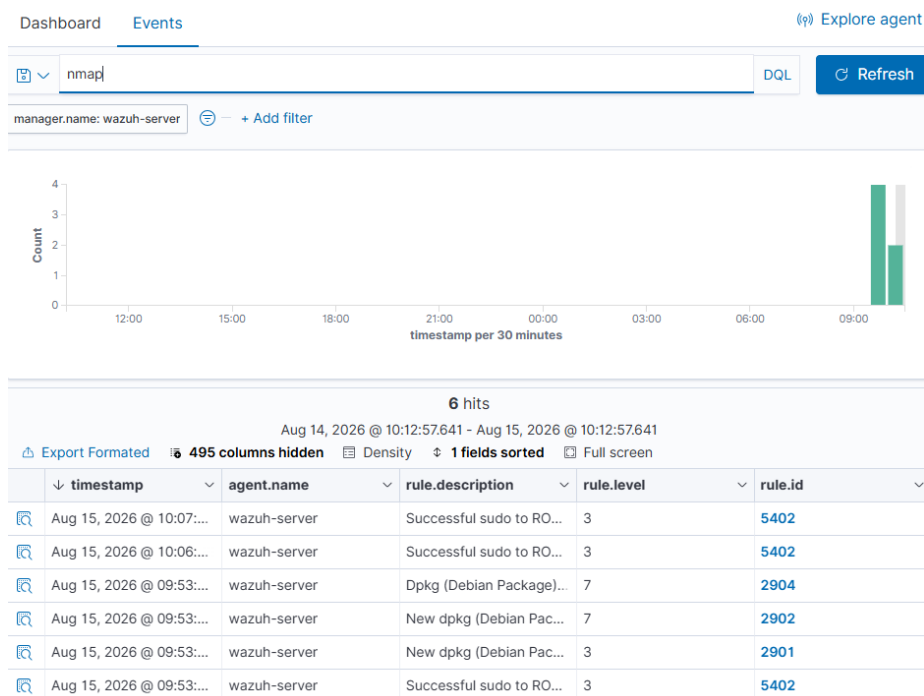
PS /home/josh/atomic-red-team/atomics> Invoke-AtomicTest T1046 -TestNumbers 12 -GetPrereqs
PathToAtomicsFolder = /home/josh/atomic-red-team/atomics

GetPrereq's for: T1046-12 Port Scan using nmap (Port range)
Attempting to satisfy prereq: Check if nmap command exists on the machine
Prereq already met: Check if nmap command exists on the machine
PS /home/josh/atomic-red-team/atomics> Invoke-AtomicTest T1046 -TestNumbers 12
PathToAtomicsFolder = /home/josh/atomic-red-team/atomics

Executing test: T1046-12 Port Scan using nmap (Port range)
Running Network Service Discovery Technique [Progress: ]

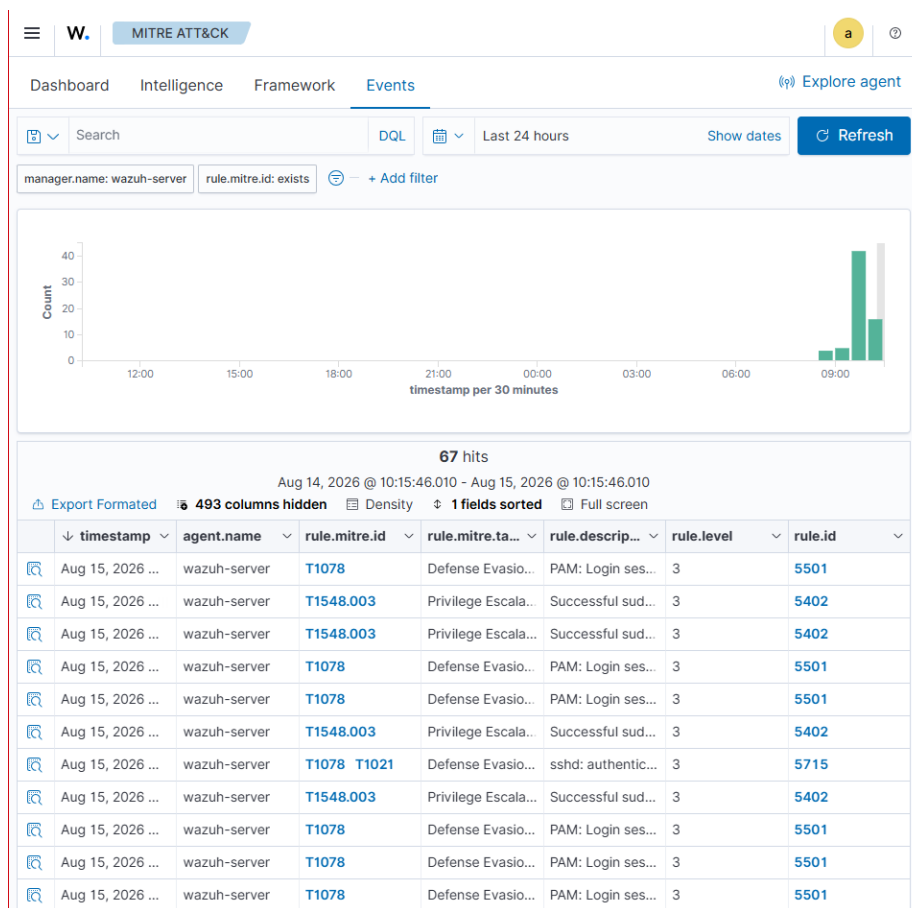
```

After the test ran, I checked the Wazuh dashboard and found six matching alerts. Wazuh had picked up the nmap package being installed and the scan itself being run with elevated (sudo) privileges.

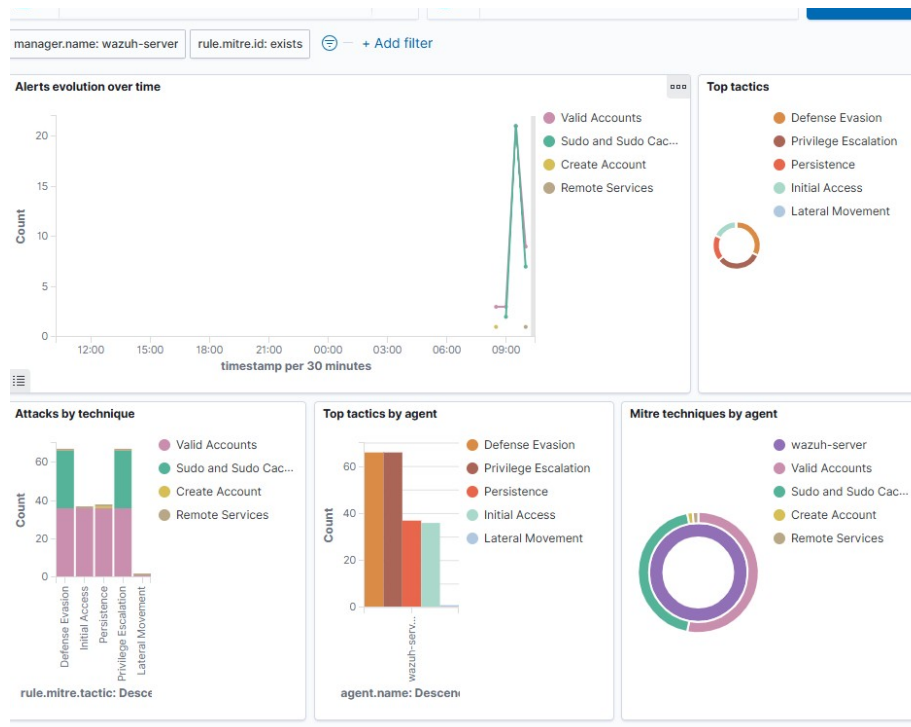


5. Mapping Detections to MITRE ATT&CK

Wazuh includes a dedicated MITRE ATT&CK module that automatically tags matching alerts with the relevant technique ID. Instead of manually figuring out what an alert means, you can see exactly which stage of an attack it corresponds to at a glance.



The module's dashboard view goes further, breaking detected activity down by tactic, the broader attacker goal behind a technique, such as Defense Evasion, Privilege Escalation, or Persistence.



6. Setting Up File Integrity Monitoring (FIM)

Next, I configured Wazuh's File Integrity Monitoring to watch the Downloads folder in real time, so that any file added, changed, or deleted there would trigger an alert immediately instead of waiting for a scheduled scan.

```
josh@wazuh-server:~/atomic-red-team/atomics$ sudo grep -A 3 "boot</directories>" /var/ossec/etc/ossec.conf
<directories>/bin,/sbin,/boot</directories>
<directories realtime="yes">/home/josh/Downloads</directories>
<!-- Files/directories to ignore -->
<ignore>/etc/mtab</ignore>
josh@wazuh-server:~/atomic-red-team/atomics$ sudo systemctl restart wazuh-manager
```

7. Connecting VirusTotal

On its own, FIM only knows that a file changed, not whether it's dangerous. To add that context, I connected a VirusTotal integration: whenever FIM detects a new file, Wazuh automatically checks that file's hash against VirusTotal's database of 70+ antivirus engines.

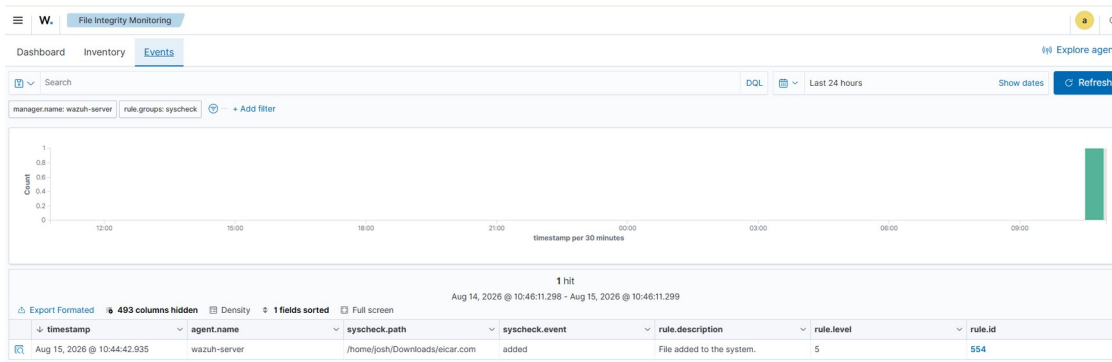
```
EOI
josh@wazuh-server:~/atomic-red-team/atomics$ sudo tail -12 /var/ossec/etc/ossec.conf
</localfile>

</ossec_config>

<ossec_config>
  <integration>
    <name>virustotal</name>
    <api_key>[REDACTED]</api_key>
    <group>syscheck</group>
    <alert_format>json</alert_format>
  </integration>
</ossec_config>
josh@wazuh-server:~/atomic-red-team/atomics$ |
```

8. Testing Detection With a Malicious File

To test the full pipeline, I dropped the EICAR test file into the Downloads folder, a standard, harmless text string that every antivirus engine recognizes and flags as a test virus, used industry-wide for exactly this kind of testing. FIM picked it up right away.



Seconds later, the VirusTotal integration returned its verdict: dozens of antivirus engines flagged the file as malicious, and Wazuh generated a high-severity alert for it.

The screenshot shows the Wazuh File Integrity Monitoring (FIM) interface with 391 hits. The table shows a hit at 10:44:50.269 on Aug 15, 2026, for agent 'wazuh-server' at path '/home/josh/Downloads/eicar.com', with rule description 'VirusTotal: Alert - /home/josh/Downloads/eicar.com ...', rule level '12', and rule id '87105'.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 15, 2026 @ 10:44:50.269	wazuh-server	VirusTotal: Alert - /home/josh/Downloads/eicar.com ...	12	87105

9. Automating the Response

Detection alone doesn't remove anything, so the last step was building an Active Response: a custom script tied specifically to Wazuh's "VirusTotal positive detection" rule, set to run automatically the moment that exact alert fires and delete the offending file.

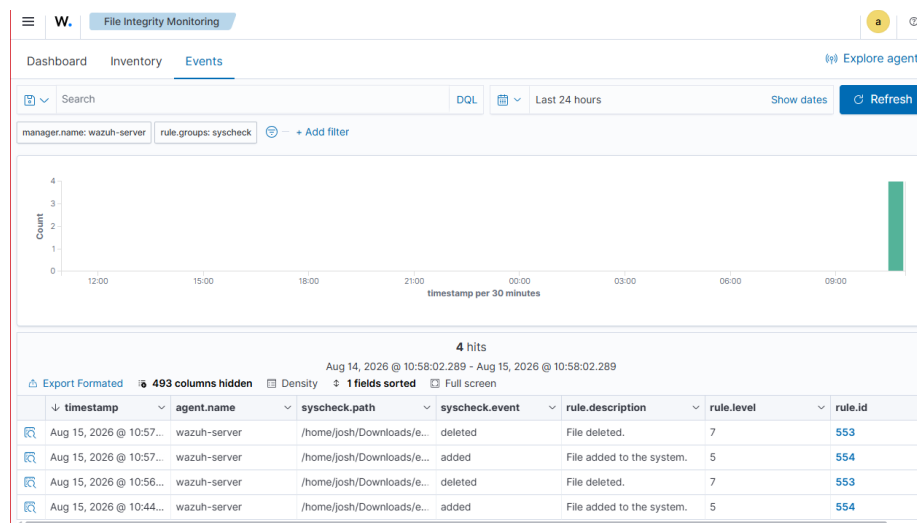
```
josh@wazuh-server:~/Downloads$ sudo tee /var/ossec/active-response/bin/remove-threat.sh > /dev/null <
< 'EOF'
#!/bin/bash
read INPUT_JSON
FILENAME=$(echo "$INPUT_JSON" | jq -r '.parameters.alert.data.virustotal.source.file')
if [ -n "$FILENAME" ] && [ -f "$FILENAME" ]; then
    rm -f "$FILENAME"
    echo "$(date) Removed malicious file: $FILENAME" >> /var/ossec/logs/active-responses.log
fi
EOF
sudo chmod 750 /var/ossec/active-response/bin/remove-threat.sh
sudo chown root:wazuh /var/ossec/active-response/bin/remove-threat.sh
josh@wazuh-server:~/Downloads$ sudo tee -a /var/ossec/etc/ossec.conf > /dev/null << 'EOF'

<ossec_config>
  <command>
    <name>remove-threat</name>
    <executable>remove-threat.sh</executable>
    <timeout_allowed>no</timeout_allowed>
  </command>

  <active-response>
    <disabled>no</disabled>
    <command>remove-threat</command>
    <location>local</location>
    <rules_id>87105</rules_id>
  </active-response>
</ossec_config>
EOF
```

10. Confirming It Works

With the active response in place, I re-triggered the test. Wazuh's File Integrity Monitoring log shows the full cycle: the file being added, and moments later, the file being deleted, confirming the automated takedown worked end to end, with no manual cleanup needed.



Summary

This lab covered the full detection lifecycle: standing up a working Wazuh SIEM, simulating real attacker techniques through Atomic Red Team, confirming those techniques were correctly detected and mapped to MITRE ATT&CK, and finally building an automated response that identifies and removes a genuine threat without manual intervention.